

SQL (Structured Query Language)

PL-SQL :-

❖ What is PL-SQL ?

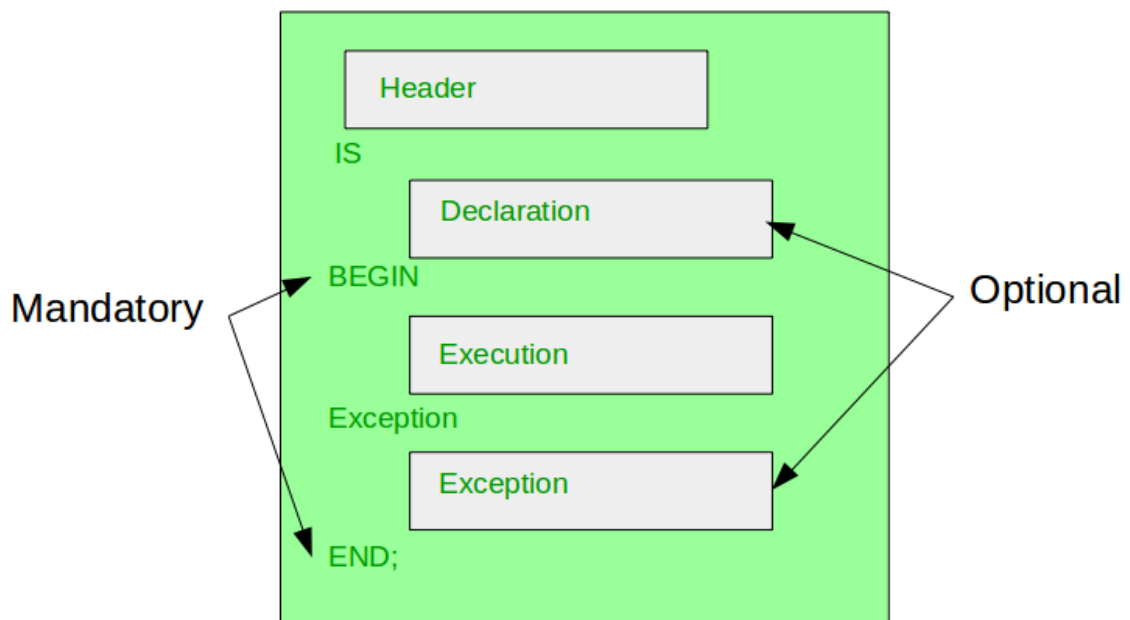
- PL/SQL stands for Procedural Language – Structured Query Language.
- It is an extension of SQL developed by Oracle Corporation.
- It allows procedural programming like loops, conditions, and variables in SQL.
- PL/SQL programs are written in the form of blocks.
- Each block has three parts – Declare, Begin, and Exception.

❖ Why use PL-SQL ?

- PL/SQL combines SQL with procedural programming features.
- It allows writing complex database logic inside the database.
- Multiple SQL statements can be executed as a single block, improving performance.
- Stored procedures and functions can be created for code reusability.

❖ Blocks in PL/SQL :-

In PL/SQL, All statements are classified into units that is called Blocks. PL/SQL blocks can include variables, SQL statements, loops, constants, conditional statements and exception handling.



- **The Declaration section:** Code block start with a declaration section, in which memory variables, constants, cursors and other oracle objects can be declared and if required initialized.

- **The Begin section:** Consist of set of SQL and PL/SQL statements, which describe processes that have to be applied to table data.
- **The Exception section:** This section deals with handling errors that arise during execution data manipulation statements, which make up PL/SQL code block.
- **The End section:** This marks the end of a PL/SQL block.

❖ Syntax of PL-SQL :-

```

DECLARE //
    <declarations section>

BEGIN
    <executable command(s)>

EXCEPTION
    <exception handling>

END; //

DECLARE ;

```

Where,

DECLARE – (Optional) Used to define variables, constants, and cursors.

BEGIN – (Mandatory) Contains the main executable code.

EXCEPTION – (Optional) Handles runtime errors.

END; – (Mandatory) Marks the end of the block.

/ – Executes the block in SQL*Plus or SQL Developer.

❖ Data Type in PL-SQL :-

PL/SQL supports several types of data to store and process values.

1. Scalar Data Types – Store a single value.

- NUMBER – Stores numeric values.
- CHAR(size) – Fixed-length character data.
- VARCHAR2(size) – Variable-length character data.
- DATE – Stores date and time values.
- BOOLEAN – Stores TRUE, FALSE, or NULL.

2. Large Object (LOB) Data Types – Store large amounts of data.

- BLOB – Binary Large Object (images, audio, video).
- CLOB – Character Large Object (large text).
- NCLOB – Unicode text.
- BFILE – Stores file locators pointing to files outside the database.

3. Composite Data Types – Group multiple values.

- RECORD – Stores related data of different types.
- TABLE – Index-by table (associative array).
- VARRAY – Variable-size array.

4. Reference Data Types

- REF CURSOR – Pointer to a query result set.

❖ Important Points in PL-SQL :-

Here are the important points for Function, Procedure, Trigger, and Cursor in PL/SQL.

1) Function :-

- A function returns a single value.
- Must have a RETURN clause.
- Can be called from SQL statements (if it does not modify data).
- Used for calculations and returning results.
- Created using CREATE [OR REPLACE] FUNCTION.

Syntax of Function :-

```
DELIMITER //  
  
CREATE [OR REPLACE] FUNCTION function_name (parameter)  
  
RETURN return_datatype  
  
BEGIN  
  
-- Executable statements  
  
RETURN return_value;  
  
END function_name; //  
  
DELIMITER ;
```

❖ Examples :- Based on Control Statements.

1) IF THEN ELSE

```
DELIMITER //
```

```
create FUNCTION checkdata (age int)
returns varchar (100)
BEGIN

    IF age>=18 THEN
        return 'Age is Valid for Voting';
    ELSE
        return 'age is invalid for Voting';

END IF; //
```

```
DELIMITER ;
```

```
SELECT checkdata(20);
Age is Valid for Voting
```

2) IF THEN ELSEIF

```
DELIMITER //
```

```
create FUNCTION result (marks int)
returns varchar (100)
BEGIN

    IF marks>=85 THEN
        return 'A+ Grade';
    ELSEIF marks>=75 THEN
        return 'A Grade';
    ELSEIF marks>=65 THEN
        return 'B+ Grade';
    ELSE
        return 'Sorry ! Fail !';

END IF;
```

```
END; //
```

```
DELIMITER ;
```

```
SELECT result(80);  
A Grade
```

3) CASE

```
DELIMITER //
```

```
create FUNCTION displaymonth (month int)  
RETURNS varchar (100)  
BEGIN  
    case month  
        WHEN 1 THEN  
            return 'Jan';  
        WHEN 2 THEN  
            return 'Feb';  
        WHEN 3 THEN  
            return 'March';  
        ELSE  
            return 'Not Match';  
    END CASE;  
END: //
```

```
DELIMITER ;
```

```
SELECT displaymonth(2);  
Feb
```